Part 5 — Kafka Administration

Topic app-events on the three Kafka brokers in the hadoop namespace. One Python producer sent 100,000 messages. One consumer group, app-events-lag, read them and printed the lag as it fell. I then deleted a broker pod, watched the leaders move, and timed how long the pod and the ISR took to come back. After the second broker came back I reassigned the replicas so each broker leads two partitions.

Topic

The tools run inside the Kafka pod. The bootstrap address is the headless service, kafka-0.kafka-hs.hadoop.svc.cluster.local:9092. The old script that points at kafka-0.kafka.hadoop.svc.cluster.local does not resolve here, so I did not use it.

```
kubectl exec -n hadoop kafka-0 -- /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 \
  --create --topic app-events \
  --partitions 6 --replication-factor 3 \
  --config min.insync.replicas=2
```

Describe shows 6 partitions and replication factor 3. The topic id is r9jxeruRT3iX7QxfVWxC6Q. min.insync.replicas=2, so a produce with acks=all still succeeds when one broker is down, because two copies remain. Before any failure each broker led two partitions, and every ISR listed all three brokers.

```
kafka-topics.sh --describe --topic app-events

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           1        1,2,0      1,2,0
1           0        0,1,2      0,1,2
2           2        2,0,1      2,0,1
3           1        1,0,2      1,0,2
4           0        0,2,1      0,2,1
5           2        2,1,0      2,1,0
```

Figure 1. app-events before the failure. 6 partitions, replication factor 3. Broker 0 leads partitions 1 and 4, broker 1 leads 0 and 3, broker 2 leads 2 and 5.

Producer

One producer, kafka-python, run from the kafka-client pod. The library is installed under /tmp/pydeps because the spark user cannot write to its home directory. acks is all, retries is 5, and the message key is the user id, so one user stays on one partition while 10,000 users still spread across the 6 partitions. The script is scripts/part5-kafka/producer.py, and the same file is on the VM at /opt/hadoop-lab/kafka/scripts/producer.py.

```
kubectl exec -n hadoop kafka-client -- bash -lc \
  'PYTHONPATH=/tmp/pydeps python3 /tmp/part5/producer.py'
```

```
python3 producer.py    # kafka-client, PYTHONPATH=/tmp/pydeps
```

```
sent 10000
sent 20000
sent 30000
sent 40000
sent 50000
sent 60000
sent 70000
sent 80000
sent 90000
RESULT sent=100000 errors=0 seconds=89.9 topic=app-events
```

Figure 2. The producer finished 100,000 messages with 0 errors in 89.9 seconds.

I sent a second batch of 20,000 after the first consumer had caught up, so the lag table had a clean window to show. The log end offsets add up to 120,000, which is those two batches.

Consumer and lag

The consumer uses group app-events-lag, turns auto-commit off, and commits after each poll. Every 5,000 messages it prints the lag, which is the end offset minus the consumer position, per partition. The script is scripts/part5-kafka/consumer.py.

The first short run printed read=0. The first poll came back empty before the group had an assignment, and the script stops on an empty poll. The next run is the one that counts. It read all 100,000 messages in 21.8 seconds. Lag started at 100,000 and the log shows it stepping down to 0. This read kept going while kafka-1 was out of the ISR, so the new leaders were serving the fetch.

```
python3 consumer.py --max-messages 100000    # group app-events-lag
```

```
read=5000 lag=95000 per_partition={4: 17090, 1: 16210, 5: 15980, 2: 16530, 0: 16990, 3: 12200}
read=25000 lag=75000 per_partition={0: 7868, 5: 9232, 2: 16530, 4: 17090, 3: 8070, 1: 16210}
read=50000 lag=50000 per_partition={0: 7868, 5: 6882, 2: 7402, 4: 7973, 3: 8070, 1: 11805}
read=75000 lag=25000 per_partition={0: 1833, 5: 0, 2: 0, 4: 7973, 3: 8070, 1: 7124}
read=100000 lag=0 per_partition={0: 0, 5: 0, 2: 0, 4: 0, 3: 0, 1: 0}
RESULT group=app-events-lag read=100000 seconds=21.8
```

Figure 3. Consumer group app-events-lag. Lag falls from 95,000 at 5,000 messages to 0 at 100,000 messages. The lines in between are the same log, kept short so the drop is visible.

After the extra 20,000 messages, kafka-consumer-groups showed lag 20,000 and no active member. The six partitions were between 3,196 and 3,440 behind. That is the lag sitting still, before the consumer joined.

```
kafka-consumer-groups.sh --describe --group app-events-lag
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID	HOST	CLIENT-ID
app-events-lag	app-events	5	15980	19176	3196	-	-	-
app-events-lag	app-events	4	17090	20508	3418	-	-	-
app-events-lag	app-events	3	17200	20640	3440	-	-	-
app-events-lag	app-events	2	16530	19836	3306	-	-	-
app-events-lag	app-events	1	16210	19452	3242	-	-	-
app-events-lag	app-events	0	16990	20388	3398	-	-	-

Figure 4. kafka-consumer-groups for app-events-lag before the second consume. Lag adds up to 20,000. CURRENT-OFFSET is the 100,000 already committed. LOG-END-OFFSET adds up to 120,000.

The same consumer then read those 20,000 in 3.8 seconds and the lag hit 0 on every partition.

```
python3 consumer.py --max-messages 20000

read=5000 lag=15000 per_partition={2: 0, 5: 1502, 1: 3242, 4: 3418, 3: 3440, 0: 3398}
read=10000 lag=10000 per_partition={2: 0, 5: 0, 1: 3162, 4: 0, 3: 3440, 0: 3398}
read=15000 lag=5000 per_partition={3: 1602, 0: 3398, 1: 0, 4: 0, 2: 0, 5: 0}
read=20000 lag=0 per_partition={3: 0, 0: 0, 2: 0, 5: 0, 1: 0, 4: 0}
RESULT group=app-events-lag read=20000 seconds=3.8
```

Figure 5. The second consume. Lag 15,000, then 10,000, then 5,000, then 0. 20,000 messages in 3.8 seconds.

Broker failure

I deleted the broker pod and let the StatefulSet start it again. kubectl delete pod -n hadoop kafka-1. Broker 1 was the leader for partitions 0 and 3.

A few seconds later those two leaders had moved. Partition 0 moved from broker 1 to broker 2. Partition 3 moved from broker 1 to broker 0. The other four leaders stayed where they were. Every ISR dropped broker 1 and kept two replicas, which is enough for min.insync.replicas=2. The topic stayed writable and the consumer kept committing, so the lag kept falling. A broker failure does not add lag. Lag only grows when the consumer stops while producers continue.

```
describe while pod kafka-1 is down

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           2        1,2,0      2,0
1           0        0,1,2      0,2
2           2        2,0,1      2,0
3           0        1,0,2      0,2
4           0        0,2,1      0,2
5           2        2,1,0      2,0
```

Figure 6. app-events while kafka-1 is down. Partitions 0 and 3 have new leaders. ISR on every partition is two brokers, and broker 1 is missing from all of them.

The new kafka-1 pod was Ready in 35 seconds. About 48 seconds after the delete, describe showed ISR 0,1,2 on every partition again, and the preferred leaders were back: partition 0 and partition 3 on broker 1. Recovery for this broker was the pod restart plus the ISR catch-up, under a minute, and the leaders returned to the preferred replica on their own.

```
describe after kafka-1 is Ready again

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           1        1,2,0      2,0,1
1           0        0,1,2      0,2,1
2           2        2,0,1      2,0,1
3           1        1,0,2      0,2,1
4           0        0,2,1      0,2,1
5           2        2,1,0      2,0,1
```

Figure 7. After kafka-1 is Ready. Leaders match Figure 1 again, and every ISR lists brokers 0, 1, and 2.

I did the same to kafka-2 so the write-up has both followers of the failure. kubectl delete pod -n hadoop kafka-2. Broker 2 led partitions 2 and 5. Partition 2 moved to broker 0, partition 5 moved to broker 1, and every ISR became 0,1.

```
describe while pod kafka-2 is down

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           1        1,2,0      0,1
1           0        0,1,2      0,1
2           0        2,0,1      0,1
3           1        1,0,2      0,1
4           0        0,2,1      0,1
5           1        2,1,0      0,1
```

Figure 8. app-events while kafka-2 is down. Partitions 2 and 5 have new leaders. ISR is 0,1 on every partition.

kafka-2 was Ready in 25 seconds, and the next describe showed ISR 0,1,2 again. The leaders did not move back to the preferred replica on their own this time. Partition 2 stayed on broker 0 and partition 5 stayed on broker 1 until the election in the next section.

Reassignment

After both brokers were back I generated a new replica order with kafka-reassign-partitions.sh and executed it. The plan puts a different broker first on each partition, two partitions each. The JSON is scripts/part5-kafka/reassign-app-events.json.

```
kubectl exec -n hadoop kafka-0 -- /opt/kafka/bin/kafka-reassign-partitions.sh \
  --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 \
  --reassignment-json-file /tmp/part5/reassign-app-events.json \
  --execute
```

Right after the reassignment the replicas were the new order, but the leader was not always the first replica. Broker 2 led nothing. Broker 0 led partitions 1, 2, and 4. Broker 1 led partitions 0, 3, and 5.

```
describe right after the reassignment, before preferred election

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           1        2,1,0      0,1,2
1           0        0,2,1      0,1,2
2           0        1,0,2      0,1,2
3           1        2,0,1      0,1,2
4           0        0,1,2      0,1,2
5           1        1,2,0      0,1,2
```

Figure 9. Replicas are the new order. The leader is the first replica only on partitions 1, 4, and 5. Broker 2 is in the replica list and the ISR, and it leads nothing yet.

Preferred election moves the leader to the first replica. The tool wants exactly one of --topic with --partition, --all-topic-partitions, or --path-to-json-file. I used --all-topic-partitions and did not also pass --topic.

```
kubectl exec -n hadoop kafka-0 -- /opt/kafka/bin/kafka-leader-election.sh \
  --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 \
  --election-type preferred --all-topic-partitions
```

After the election every leader is the first replica, and each broker leads two partitions. ISR is 0,1,2 on all six. This is still the topic today. The consumer group is caught up: current offset equals log end offset on every partition, lag is 0, and the group has no active member.

```
describe after preferred leader election

Topic: app-events    TopicId: r9jxeruRT3iX7QxfVWxC6Q
PartitionCount: 6    ReplicationFactor: 3    Configs: min.insync.replicas=2,segment.bytes=1073741824

Partition   Leader   Replicas   ISR
0           2        2,1,0      0,1,2
1           0        0,2,1      0,1,2
2           1        1,0,2      0,1,2
3           2        2,0,1      0,1,2
4           0        0,1,2      0,1,2
5           1        1,2,0      0,1,2
```

Figure 10. After preferred election. Broker 0 leads 1 and 4, broker 1 leads 2 and 5, broker 2 leads 0 and 3.

```
kafka-consumer-groups.sh --describe --group app-events-lag

GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
app-events-lag  app-events      5          19176           19176           0               -               -               -
app-events-lag  app-events      4          20508           20508           0               -               -               -
app-events-lag  app-events      3          20640           20640           0               -               -               -
app-events-lag  app-events      2          19836           19836           0               -               -               -
app-events-lag  app-events      1          19452           19452           0               -               -               -
app-events-lag  app-events      0          20388           20388           0               -               -               -
```

Figure 11. app-events-lag now. Lag is 0 on all 6 partitions. The dashes under CONSUMER-ID mean nobody is in the group.

Where each broker stands after the election:

Partition	Leader	Replicas	ISR
0	2	2,1,0	0,1,2
1	0	0,2,1	0,1,2
2	1	1,0,2	0,1,2
3	2	2,0,1	0,1,2
4	0	0,1,2	0,1,2
5	1	1,2,0	0,1,2

Where to look

Scripts on the VM: /opt/hadoop-lab/kafka/scripts/producer.py, consumer.py, and reassign-app-events.json.

The describe and lag captures from the test are on the VM in /tmp/part5/.

Describe the topic: kubectl exec -n hadoop kafka-0 -- /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 --describe --topic app-events

Lag: kubectl exec -n hadoop kafka-0 -- /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka-0.kafka-hs.hadoop.svc.cluster.local:9092 --describe --group app-events-lag